Exercise 7: Financial Forecasting

Scenario: You are developing a financial forecasting tool that predicts future values based on past data.

Objective: The objective of this exercise is to understand and implement recursion to predict future values based on past growth patterns.

1. Understanding Recursive Algorithms

Recursion is a programming technique where a method calls itself to solve smaller subproblems of the same problem until it reaches a base condition.

It helps in:

- Breaking complex problems into simpler ones
- Reducing code complexity
- Solving problems like mathematical sequences, tree traversal, and forecasting models

Base Case: The condition where recursion stops
Recursive Case: The function calls itself with smaller input

2. Setup

A recursive method is created to calculate future values based on growth rate and number of years.

We assume:

- initial value
- growth rate
- number of years

3. Implementation

Recursive Algorithm for Financial Forecasting

```
public class FinancialForecast {

public static double forecast(double value, double rate, int years) {

if(years == 0) {
return value;
}

return forecast(value * (1 + rate), rate, years - 1);
}

public static void main(String[] args) {

double initialValue = 10000;
double growthRate = 0.10;
int years = 5;
```

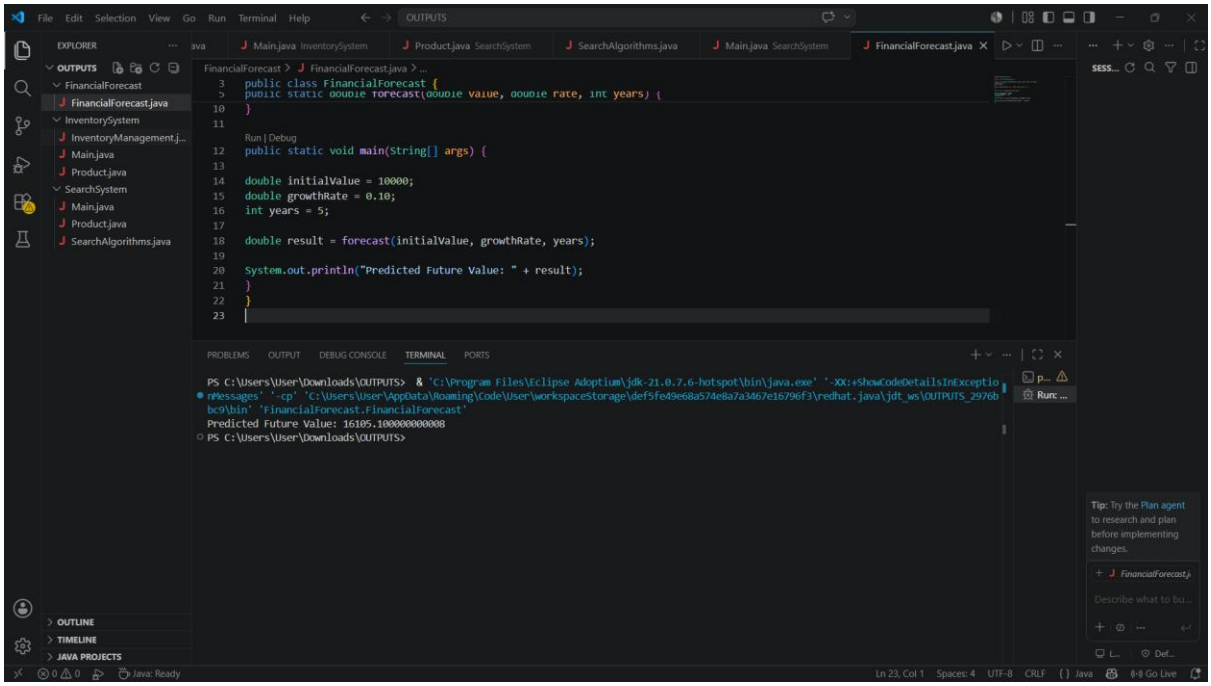
```
double result = forecast(initialValue, growthRate, years);

System.out.println("Predicted Future Value: " + result);
}
}
```

5. Output Screenshot

output :

Predicted Future Value: 16105.1



6.Analysis

Time Complexity:

The recursive solution calls itself once per year, so time complexity is O(n), where n is the number of years.

Space Complexity:
O(n) due to recursion stack usage.

Optimization:

The recursive solution can be optimized using:

- Memoization (storing repeated results)
- Iterative approach (loop instead of recursion)

Iterative version avoids recursion overhead and is more efficient for large inputs.

7. Conclusion

Recursion simplifies the financial forecasting logic by breaking the problem into smaller steps. However, for large datasets, iterative or memoized solutions are preferred for better performance.